**Part A**

**Ex: 1**

**Title:** Program that uses simple technique to a) search a key b) to sort n elements**.**

**Problem Description:** Run the program for varied values of n>5000 and record the time taken to search/sort. Plot a graph of the time taken v/s n. The elements can be read from a file or can be generated using the random number generator.

**Method:** A Brute Force- Non recursive algorithm implementation. Sequential Search and Selection Sort.

**Theory Reference:** Module 1

***Algorithm***

## Sequential Search Algorithm (Linear Search)

**Input:** An array `a` of size `n`, and a key to search.
**Output:** The index of the key if found, otherwise -1.

**Steps:**

1. **Start**
2. Loop through each element of the array from `i = 0` to `i = n-1`:
   - If `a[i] == key`, return `i` (key found at index `i`).
3. If the loop completes without finding the key, return `-1` (key not found).
4. **End**

## Selection Sort Algorithm

**Input:** An array `a` of size `n`.
**Output:** The sorted array in ascending order.

**Steps:**

1. **Start**
2. Loop through each element from `i = 0` to `n-2`:
   - Assume `i` as the `min`.
   - Loop through the remaining unsorted part from `j = i+1` to `n-1`:
     - If `a[j] < a[min]`, update `min = j`.
   - Swap `a[i]` with `a[min]` (placing the smallest element at the correct position).
3. Repeat until the entire array is sorted.
4. **End**

**ALGORITHM** *SelectionSort*$(A[0..n-1])$

```
//Sorts a given array by selection sort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in ascending order
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min]   min ← j
    swap A[i] and A[min]
```

As an example, the action of the algorithm on the list 89, 45, 68, 90, 29, 34, 17 is illustrated below in Figure.

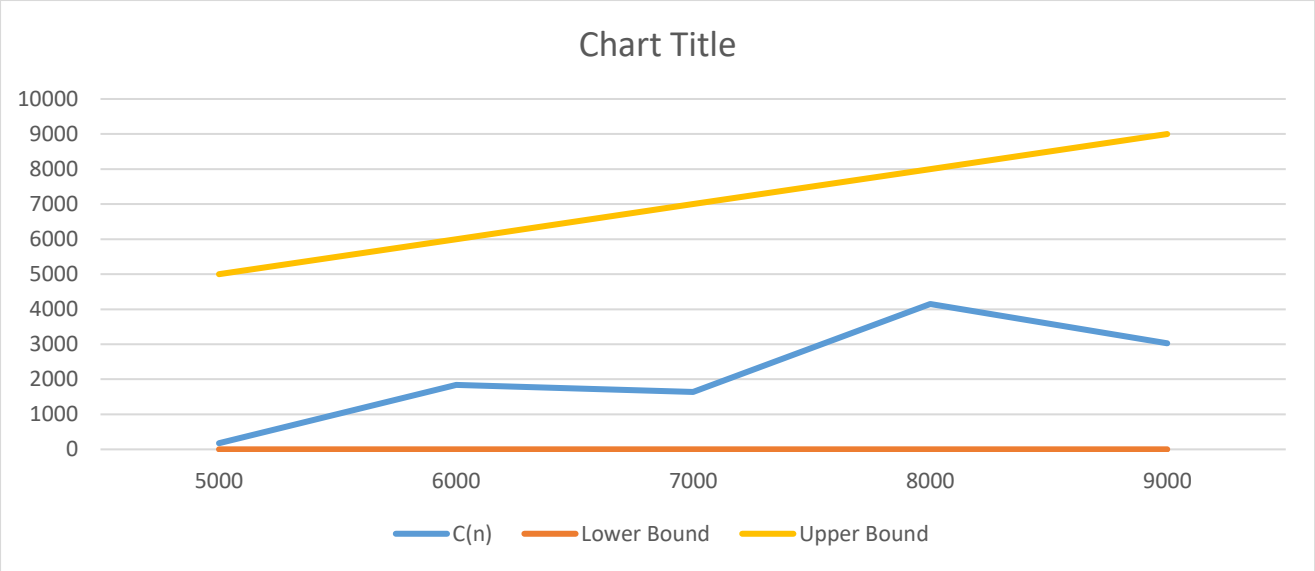
| | | | | | | |
|---|---|---|---|---|---|---|
| \| 89 | 45 | 68 | 90 | 29 | 34 | **17** |
| 17 \| | 45 | 68 | 90 | **29** | 34 | 89 |
| 17 | 29 \| | 68 | 90 | 45 | **34** | 89 |
| 17 | 29 | 34 \| | 90 | **45** | 68 | 89 |
| 17 | 29 | 34 | 45 \| | 90 | **68** | 89 |
| 17 | 29 | 34 | 45 | 68 \| | 90 | **89** |
| 17 | 29 | 34 | 45 | 68 | 89 \| | 90 |

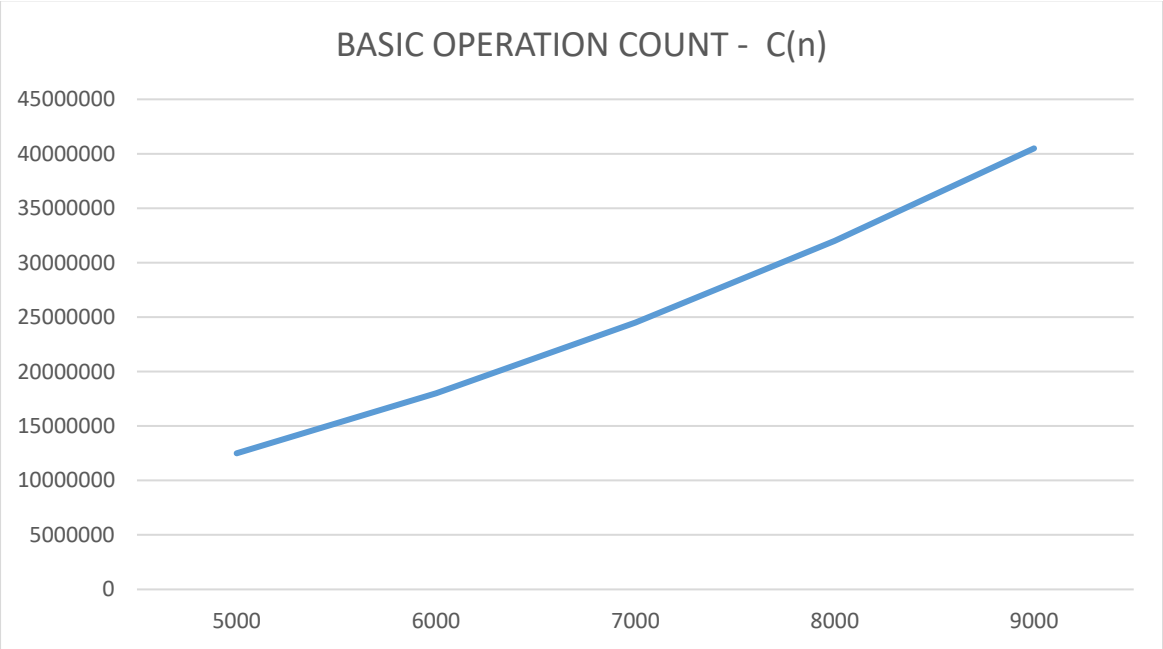
Example of sorting with selection sort. Each line corresponds to one iteration of the algorithm, i.e., a pass through the list tail to the right of the vertical bar; an element in bold indicates the smallest element found. Elements to the left of the vertical bar are in their final positions and are not considered in this and subsequent iterations.

The analysis of selection sort is straightforward. The input's size is given by the number of elements n; the algorithm's basic operation is the key comparison A[j] < A[min]. The number of times it is executed depends only on the array's size.

| INPUT SIZE (N) | BASIC OPERATION COUNT | LOWE BOUND (BEST CASE 1) | UPPER BOUND (WORST CASE – n ) |
|---|---|---|---|
| 5000 | 176 | 1 | 5000 |
| 6000 | 1837 | 1 | 6000 |
| 7000 | 1634 | 1 | 7000 |
| 8000 | 4149 | 1 | 8000 |
| 9000 | 3028 | 1 | 9000 |

**Sequential Search**



**Selection Sort**

| INPUT SIZE (N) | BASIC OPERATION COUNT - C(n) – depends only n |
|---|---|
| 5000 | 12497500 |
| 6000 | 17997000 |
| 7000 | 24496500 |
| 8000 | 31996000 |
| 9000 | 40495500 |